

PX4 TRACK 目标指向飞行模式设计报告

1. 设计目标

在 PX4 中增加独立的 **TRACK 飞行模式**，使其与 Stabilized、Altitude、Position、Hold 等模式并列，并能够通过遥控器模式开关直接切换。

TRACK 模式下：

1. 视觉模块读取目标在图像中的位置，计算目标方向；
2. 飞控自动调整飞行器姿态，使摄像头光轴持续指向目标；
3. 油门杆控制飞行器总推力大小；
4. 原 Yaw 通道改为控制机身绕摄像头光轴的旋转，即 twist；
5. PX4 原有姿态环、角速度环和控制分配模块继续负责底层稳定和电机输出。

第一阶段不引入位置、速度或高度闭环，因此油门杆控制的是**归一化总推力**，而不是高度或前向速度。各电机的具体输出仍由 PX4 姿态控制、角速度控制和 Control Allocation 自动完成。

2. 坐标系与控制定义

PX4 多旋翼机体系采用 FRD 坐标系：

- X：机头向前；
- Y：机体向右；
- Z：机体向下。

摄像头沿机体向上的 Z 方向安装，因此其中心光轴近似为机体系的：

```
camera_axis_body = [0, 0, -1]
```

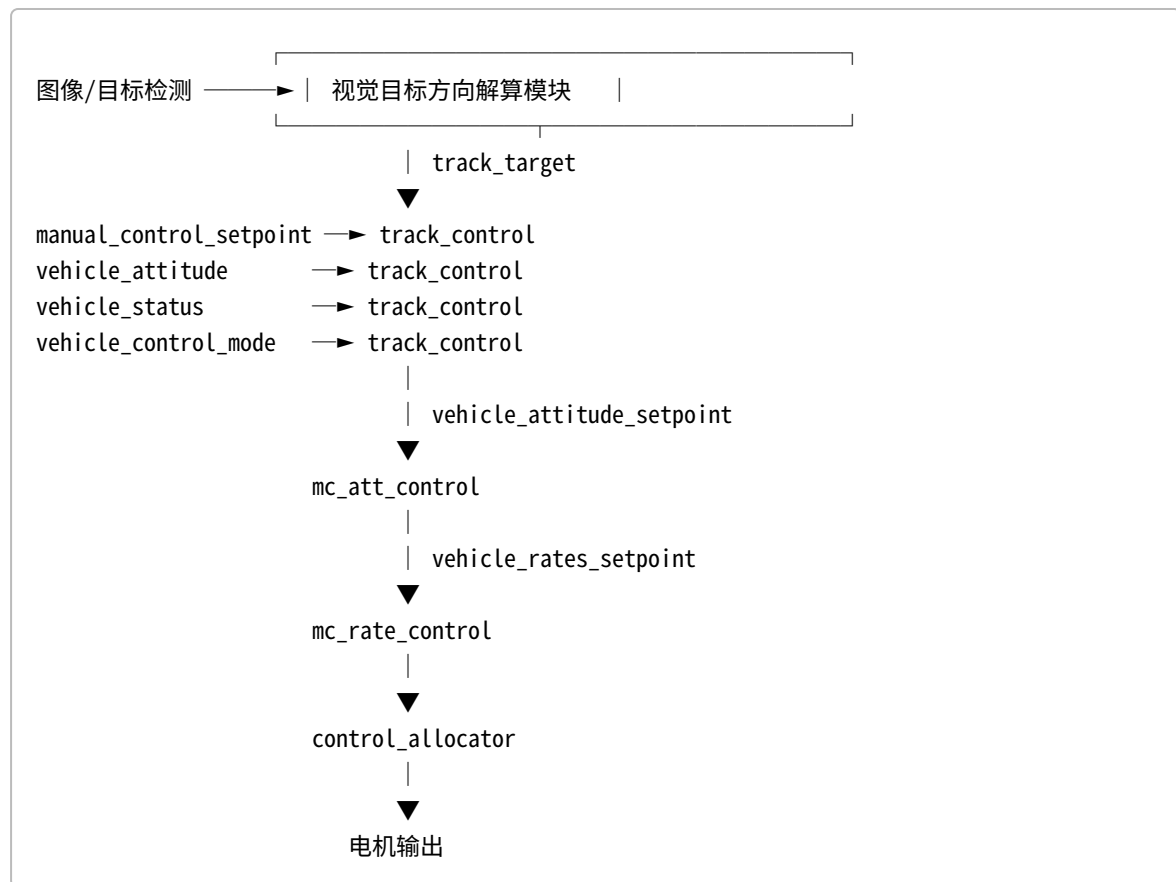
目标居于图像中心时，目标视线应与机体 **-Z** 轴重合。`vehicle_attitude` 中的四元数描述机体系 FRD 到地理系 NED 的旋转。

TRACK 模式中的三个控制自由度定义如下：

自由度	控制来源
摄像头光轴两个方向的倾斜	视觉目标方向
绕摄像头光轴的 twist	遥控器 Yaw 通道
推力大小	遥控器油门通道

Roll、Pitch 摇杆第一版不参与直接姿态控制。后续可将其扩展为目标图像偏置、人工修正或退出 TRACK 的触发输入。

3. 总体控制架构



新增的 `track_control` 只负责：

- 目标视线处理；
- 期望姿态构造；
- twist 控制；
- 油门到总推力的映射；
- 目标丢失和模式切换保护。

不建议在 `track_control` 中重新实现姿态 PID、角速度 PID 或电机混控。

4. TRACK 飞行模式接入

4.1 增加导航状态

修改：

```
msg/VehicleStatus.msg
```

PX4 v1.16 的 `VehicleStatus.msg` 中存在预留导航状态编号，例如 `NAVIGATION_STATE_FREE3 = 9`。建议将该预留值用于 TRACK：

```
uint8 NAVIGATION_STATE_TRACK = 9
```

不要改变后续已有导航状态的编号，以免影响日志、MAVLink 状态映射和参数兼容性。

这里的 9 是内部 `nav_state` 编号，不是 `COM_FLTMODE` 参数值，两者必须区分。

4.2 增加遥控器模式槽选项

修改：

```
src/modules/commander/module.yaml
```

在 `COM_FLTMODE1` ~ `COM_FLTMODE6` 的枚举列表中增加：

```
16: Track
```

建议使用参数值 16，避免和 v1.16 已有的模式参数值冲突。

随后修改：

```
src/modules/manual_control/ManualControl.cpp
```

在模式参数到导航状态的映射函数中增加：

```
case 16:  
    return vehicle_status_s::NAVIGATION_STATE_TRACK;
```

PX4 的模式槽参数通过 `ManualControl` 转换成目标导航状态，再由 Action Request 请求 Commander 切换模式。

配置完成后，可以在 QGroundControl 中将某个遥控器模式槽设置为 Track，再通过遥控器开关切换。

4.3 设置控制模式标志

修改：

```
src/modules/commander/ModeUtil/control_mode.cpp
```

增加：

```
case vehicle_status_s::NAVIGATION_STATE_TRACK:
    vehicle_control_mode.flag_control_manual_enabled = true;
    vehicle_control_mode.flag_control_attitude_enabled = true;
    vehicle_control_mode.flag_control_rates_enabled = true;
    vehicle_control_mode.flag_control_allocation_enabled = true;
    break;
```

以下标志保持关闭：

```
flag_control_position_enabled
flag_control_velocity_enabled
flag_control_altitude_enabled
flag_control_climb_rate_enabled
flag_control_acceleration_enabled
flag_control_auto_enabled
flag_control_offboard_enabled
```

这表示：

- 模式仍依赖遥控器输入；
- 启用姿态控制器；
- 启用角速度控制器；
- 启用控制分配；
- 不运行位置和高度控制器。

PX4 正是在 `control_mode.cpp` 中根据导航状态设置各级控制器使能标志。

4.4 增加模式进入条件

修改：

```
src/modules/commander/ModeUtil/mode_requirements.cpp
```

TRACK 至少需要：

- 有效姿态估计；
- 有效角速度估计；
- 有效遥控器输入；
- 多旋翼姿态和角速度控制器正常。

第一阶段建议增加以下限制：

```
禁止直接在 TRACK 模式解锁
```

推荐操作流程为：

Stabilized/Position 模式解锁并起飞



切换到 TRACK

这样可避免地面解锁时视觉目标突变造成姿态跳转。PX4 的模式要求系统本身支持按导航状态配置姿态、位置、手动输入和禁止解锁等条件。

后续完成目标有效性检查和姿态渐入后，再考虑允许 TRACK 模式直接解锁。

4.5 模式名称和地面站显示

需要全局搜索并检查以下内容：

```
NAVIGATION_STATE_FREE3
navigation state name
nav_state_names
PX4_CUSTOM_MAIN_MODE
PX4_CUSTOM_SUB_MODE
```

内部 RC 模式切换只需要 Commander 能识别 `NAVIGATION_STATE_TRACK`。

但如果要求：

- QGroundControl 顶部明确显示 “Track” ；
- 通过 MAVLink `SET_MODE` 选择 TRACK；
- 在飞行日志中使用完整模式名称；

则还需要补充 PX4 Custom Mode 映射，并视情况修改 QGroundControl 的 PX4 Firmware Plugin。

第一阶段可以先实现遥控器切换和内部状态，地面站显示可放在第二阶段。

5. 新增 track_control 模块

5.1 建议目录

```
src/modules/track_control/
├── CMakeLists.txt
├── Kconfig
├── TrackControl.cpp
├── TrackControl.hpp
├── track_control_params.c
└── module.yaml
```

模块名称：

```
track_control
```

板级配置中增加：

```
CONFIG_MODULES_TRACK_CONTROL=y
```

实际文件位于对应飞控板的：

```
boards/<vendor>/<board>/default.px4board
```

5.2 模块订阅消息

消息	用途
<code>vehicle_status</code>	判断当前是否处于 TRACK、是否解锁
<code>vehicle_control_mode</code>	判断姿态控制链是否启用
<code>manual_control_setpoint</code>	读取油门和 Yaw 摇杆
<code>vehicle_attitude</code>	当前姿态和坐标变换
<code>track_target</code>	视觉模块发布的目标方向
<code>vehicle_land_detected</code>	地面状态和推力限制
<code>parameter_update</code>	参数在线更新

`manual_control_setpoint` 中的摇杆量通常归一化到 `[-1,1]`，并带有输入有效性和数据源信息。

5.3 模块发布消息

必须发布：

```
vehicle_attitude_setpoint
```

建议额外发布：

```
track_status
```

用于日志记录、调试和地面站状态显示。

`vehicle_attitude_setpoint` 包含目标四元数、推力向量和 Yaw 前馈等字段。多旋翼通常只使用机体 Z 轴推力，且向上的推力在 FRD 坐标系中表现为负 Z。

5.4 模块运行频率

建议由 `vehicle_attitude` 更新触发：

100~250 Hz

视觉目标可以仅以 20~60 Hz 更新。两帧视觉数据之间，TRACK 模块保持并滤波最近一次有效目标方向。

所有目标数据必须根据 `timestamp_sample` 进行超时检查，不能长期使用旧图像结果。

6. 自定义 uORB 消息

6.1 TrackTarget.msg

建议视觉模块不要只向控制器传递像素坐标，而是传递已经完成相机内参和安装外参转换的**机体系目标方向**。

```
uint64 timestamp
uint64 timestamp_sample

bool valid

uint8 TRACKING_STATE_NONE = 0
uint8 TRACKING_STATE_DETECTED = 1
uint8 TRACKING_STATE_TRACKING = 2
uint8 TRACKING_STATE_PREDICTED = 3
uint8 tracking_state

uint32 target_id
uint32 sequence

float32 confidence

# 从摄像头位置指向目标的单位向量
# 坐标系为PX4机体FRD：
# X向前、Y向右、Z向下
# 摄像头正上方中心目标应接近 [0, 0, -1]
float32[3] direction_body

# 以下字段主要用于日志和调试
# 归一化图像坐标，建议范围[-1, 1]
float32 image_x
float32 image_y
```

```
# 归一化目标尺寸；未知时填NAN
float32 target_size
```

这样做的优点是：

- TRACK 控制器不需要知道相机分辨率；
- 不需要在飞控控制模块内保存相机内参；
- 摄像头安装外参由视觉模块统一处理；
- 更换摄像头或检测算法不会影响控制器。

视觉模块内部转换过程为：

```
图像像素
  ↓ 相机内参反投影
相机坐标系目标方向
  ↓ 相机到机体安装外参
direction_body
```

若视觉算法运行在伴随计算机上，可以通过 uXRCE-DDS 或自定义 MAVLink 消息将数据送入飞控，再统一转换成内部 `track_target` 话题。

6.2 TrackStatus.msg

建议增加调试状态消息：

```
uint64 timestamp

bool active
bool target_valid
bool attitude_limited
bool failsafe_active

uint8 failure_reason

uint32 target_age_us

float32[3] filtered_direction_body
float32[4] attitude_setpoint_q

float32 twist_angle
float32 twist_rate
float32 thrust_setpoint
```

该消息不参与控制，仅用于：

- ULog 记录；
- 检查目标是否过期；

- 检查姿态设定是否连续；
- 分析推力和 twist 输入；
- 判断限制器是否介入。

7. TRACK 控制逻辑

7.1 模式进入

检测到：

```
vehicle_status.nav_state == NAVIGATION_STATE_TRACK
```

且上一周期不在 TRACK 时，执行初始化：

1. 检查遥控器输入有效；
2. 检查目标数据有效且未超时；
3. 将当前姿态作为初始目标姿态；
4. 根据当前机身朝向初始化 twist 基准；
5. 清除目标方向滤波器；
6. 清除目标丢失计时器；
7. 启动姿态渐入过程。

进入时不能直接从当前姿态跳到视觉计算姿态，建议在 0.3~1.0 s 内做球面插值：

```
q_command = slerp(q_entry, q_track, blend_ratio)
```

以防模式切换瞬间出现过大的角速度指令。

7.2 目标方向转换

视觉模块输出机体系目标方向：

```
d_B = normalize(direction_body)
```

利用当前姿态转换到 NED：

```
d_N = R_NB(q_current) · d_B
```

由于摄像头光轴为机体 $-Z$ ，期望机体正 Z 轴方向为：

```
z_d = -d_N
```

因此最终满足：

```
-Z_B 指向目标
```

视觉目标一旦更新，就保存其 NED 方向。这样即使视觉帧率低于姿态控制频率，飞控也能在姿态变化过程中保持一个相对稳定的目标空间方向。

第一版可以忽略目标和飞行器在单帧间隔内的相对运动；高速跟踪时再加入时间同步和延迟补偿。

7.3 构造无 twist 基准姿态

仅给定 `z_d` 只能确定两个姿态自由度，还缺少绕 `z_d` 的旋转。

建议保存上一周期目标姿态的 X 轴 `x_prev`，并将其投影到与 `z_d` 垂直的平面：

```
x_base = x_prev - z_d * dot(z_d, x_prev)
x_base = normalize(x_base)
```

随后计算：

```
y_base = normalize(z_d × x_base)
```

这种方法相当于在目标视线变化时尽可能保持原有 twist，能降低机身无意义旋转。

如果 `x_prev` 与 `z_d` 接近平行，投影长度过小，则改用：

- 上一周期 Y 轴；
- NED 北向；
- 或当前机身 X 轴；

作为备用参考方向。

7.4 Yaw 摇杆控制 twist

对 Yaw 摇杆进行死区和指数处理：

```
yaw_input = expo_deadzone(yaw_stick)
```

转换为绕目标视线的旋转速度：

```
twist_rate = yaw_input × TRK_TWIST_MAX
```

积分得到 twist 角：

```
twist_angle += twist_rate × dt
```

再将基准 X、Y 轴绕 `z_d` 旋转：

```
x_d = RotateAroundAxis(x_base, z_d, twist_angle)  
y_d = z_d × x_d
```

构造目标旋转矩阵：

```
R_d = [x_d y_d z_d]
```

最后转换为：

```
q_d = Quaternion(R_d)
```

需要特别强调：

```
twist ≠ 普通地理系Yaw
```

普通 Yaw 是绕 NED 地理竖直轴转动，而 TRACK twist 是绕当前目标视线和摄像头光轴转动。当飞机大角度倾斜时，两者明显不同。

因此不应把 Yaw 摇杆直接写入标准的：

```
yaw_sp_move_rate
```

第一版建议：

```
yaw_sp_move_rate = 0
```

将 twist 完整编码进目标四元数 `q_d`。

7.5 油门到总推力映射

油门输入使用：

```
manual_control_setpoint.throttle
```

建议复用 `mc_att_control` 当前使用的油门曲线思想：

- 最低推力；
- 悬停推力；
- 最大推力；
- 落地和电机启动阶段的推力渐变。

PX4 v1.16 的 `mc_att_control` 已实现基于 `MPC_THR_CURVE`、`MPC_THR_HOVER` 和 `MPC_THR_MAX` 的手动油门映射。

TRACK 输出：

```
attitude_sp.thrust_body[0] = 0.0f;  
attitude_sp.thrust_body[1] = 0.0f;  
attitude_sp.thrust_body[2] = -thrust_setpoint;
```

这意味着油门杆控制总推力大小，而推力方向由目标姿态决定。

建议将公共油门曲线抽成共享函数，避免 TRACK 和 Stabilized 两套实现逐渐产生差异。第一版也可以在 `track_control` 中复制相同映射逻辑，但必须保持参数和限幅一致。

7.6 发布姿态设定

最终发布：

```
vehicle_attitude_setpoint_s attitude_sp{};  
  
q_d.copyTo(attitude_sp.q_d);  
  
attitude_sp.thrust_body[0] = 0.0f;  
attitude_sp.thrust_body[1] = 0.0f;  
attitude_sp.thrust_body[2] = -thrust_setpoint;  
  
attitude_sp.yaw_sp_move_rate = 0.0f;  
attitude_sp.reset_integral = reset_integral;  
attitude_sp.timestamp = hrt_absolute_time();  
  
_vehicle_attitude_setpoint_pub.publish(attitude_sp);
```

`mc_att_control` 接收该消息，计算姿态误差并输出 `vehicle_rates_setpoint`，底层角速度控制和控制分配流程保持不变。

8. 解决 mc_att_control 设定值冲突

这是必须处理的修改。

PX4 v1.16 的 `mc_att_control` 在以下条件同时满足时，会根据遥控器 Roll、Pitch、Yaw 自动生成 Stabilized 姿态设定：

```
flag_control_manual_enabled == true
flag_control_altitude_enabled == false
flag_control_velocity_enabled == false
flag_control_position_enabled == false
```

TRACK 恰好也具有这组控制标志，因此如果不修改，会出现：

```
track_control 发布视觉姿态设定
      与
mc_att_control 发布普通手动姿态设定
```

相互覆盖的问题。源码随后又会读取 `vehicle_attitude_setpoint`，因此两个设定源会形成竞争。

建议在 `mc_att_control` 中保存当前导航状态：

```
uint8_t _nav_state{vehicle_status_s::NAVIGATION_STATE_MAX};
```

读取 `vehicle_status` 时更新：

```
_nav_state = vehicle_status.nav_state;
```

修改自动生成姿态设定的条件：

```
const bool generate_manual_attitude_sp =
    !_vehicle_control_mode.flag_control_manual_enabled
    && !_vehicle_control_mode.flag_control_altitude_enabled
    && !_vehicle_control_mode.flag_control_velocity_enabled
    && !_vehicle_control_mode.flag_control_position_enabled
    && (_nav_state != vehicle_status_s::NAVIGATION_STATE_TRACK);

if(generate_manual_attitude_sp) {
    generate_attitude_setpoint(q, dt);
}
```

这样可以保持：

```
flag_control_manual_enabled = true
```

准确表达 TRACK 使用遥控器输入，同时保证 TRACK 下只有 `track_control` 发布有效姿态设定。

一个侵入性更小但语义较差的替代办法，是在 TRACK 中将 `flag_control_manual_enabled` 设为 false，并由模块直接读取遥控器。第一版不推荐，因为它会掩盖该模式实际依赖 RC 输入的事实。

9. 目标丢失处理

不能在目标丢失时突然将推力设为零。

建议采用分级策略。

9.1 短时丢失

当目标年龄小于：

```
TRK_TGT_TIMEOUT
```

保持最近一次目标方向，并继续使用姿态滤波。

推荐初始值：

```
TRK_TGT_TIMEOUT = 0.30 s
```

9.2 超时失效

超过目标超时时间后：

1. 停止更新目标方向；
2. 冻结当前期望姿态；
3. 保持油门杆对总推力的控制；
4. 进入目标丢失状态；
5. 请求切换到安全模式。

推荐默认回退模式：

```
Stabilized
```

因为 Stabilized 只依赖姿态估计和遥控器，不要求位置估计。

如果已确认本地位置有效，也可以配置为：

Position/Hold

但不能无条件切到 Hold，否则在无定位或定位失效时可能引入新的模式拒绝或失控风险。

建议失效流程：

0~0.3 s：保持最近目标
0.3~0.8 s：冻结姿态并等待目标恢复
超过0.8 s：请求切换至Stabilized

目标重新出现时，不应瞬间恢复追踪，而应重新执行短时间姿态渐入。

10. 姿态和推力限制

建议至少增加以下保护：

限制	作用
最大目标倾角	防止目标位于极端侧向或下方时翻转
最大姿态设定角速度	防止图像跳变产生瞬时大指令
目标方向低通滤波	降低检测框抖动
最小目标置信度	拒绝不可靠目标
最大目标数据年龄	防止使用旧帧
推力上下限	防止油门映射异常
模式进入姿态渐变	防止切换冲击

需要注意：限制最大倾角后，摄像头可能不能始终将目标严格保持在图像中心。这是安全限制与目标指向精度之间必须接受的权衡。

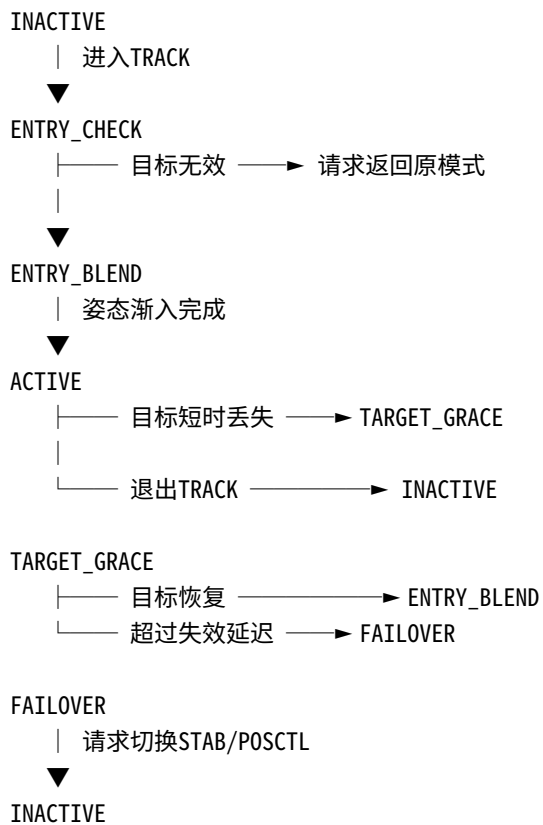
11. 建议参数

参数	建议初值	说明
TRK_TGT_TIMEOUT	0.30 s	目标数据超时
TRK_LOST_DELAY	0.80 s	请求退出 TRACK 的延迟
TRK_CONF_MIN	0.50	最低目标置信度
TRK_TWIST_MAX	90°/s	最大 twist 速度
TRK_TWIST_DZ	0.05	Yaw 摇杆死区

参数	建议初值	说明
TRK_DIR_TC	0.08 s	目标方向滤波时间常数
TRK_ATT_RATE_MAX	120°/s	目标姿态变化率上限
TRK_TILT_MAX	60°	相对地理竖直方向最大倾角
TRK_ENTRY_T	0.50 s	模式进入姿态渐变时间
TRK_LOST_ACT	STAB	目标丢失后的动作
TRK_THR_MIN	沿用系统值	最小总推力
TRK_THR_MAX	沿用系统值	最大总推力

推力相关参数优先复用 PX4 已有 `MPC_THR_*` 参数，避免重复配置。

12. 推荐状态机



13. 主要源码修改清单

文件或目录	修改内容
msg/VehicleStatus.msg	增加 NAVIGATION_STATE_TRACK
msg/TrackTarget.msg	新增目标方向消息
msg/TrackStatus.msg	新增调试状态消息
src/modules/commander/module.yaml	在 COM_FLTMODE* 中增加 Track
src/modules/manual_control/ManualControl.cpp	模式参数值映射到 TRACK
src/modules/commander/ModeUtil/control_mode.cpp	设置 TRACK 控制器使能标志
src/modules/commander/ModeUtil/mode_requirements.cpp	增加 TRACK 进入要求
src/modules/mc_att_control/	禁止 TRACK 下生成普通手动姿态设定
src/modules/track_control/	新增完整 TRACK 控制模块
对应 default.px4board	编译进 track_control
模式名称转换相关文件	增加 Track 字符串
MAVLink Custom Mode 相关文件	可选，支持地面站选择与显示
QGroundControl PX4 插件	可选，显示正式 Track 模式名称

修改 VehicleStatus.msg 和增加 uORB 消息后，需要重新生成消息头文件并完整编译固件。

14. 推荐实现顺序

第一阶段：打通模式框架

1. 增加 NAVIGATION_STATE_TRACK；
2. 增加 COM_FLTMODE 映射；
3. 增加 Commander 控制标志；
4. 新建 track_control 空模块；
5. 确认遥控器可在 TRACK 和 Stabilized 间切换；
6. 确认日志中的 nav_state 正确。

第二阶段：固定姿态输出

1. TRACK 发布当前姿态；
2. 油门控制总推力；
3. 确认 mc_att_control 不再生成普通手动姿态；
4. 确认只有一个有效姿态设定源；
5. 桨叶拆除状态下检查电机控制量。

第三阶段：接入视觉目标

1. 注入固定 `direction_body`；
2. 验证目标偏左、偏右、偏前、偏后时的姿态方向；
3. 验证图像中心对应 `[0,0,-1]`；
4. 加入目标方向滤波和姿态限速；
5. 验证坐标轴和符号。

第四阶段：加入 twist

1. Yaw 摇杆积分为 twist；
2. 验证机身旋转时光轴仍指向目标；
3. 验证大倾角下没有按地理 Yaw 旋转；
4. 验证松杆后 twist 角保持；
5. 验证模式重新进入时没有机身突转。

第五阶段：失效保护

1. 目标短时丢失；
2. 目标长期丢失；
3. RC 丢失；
4. 目标方向突然跳变；
5. 模式快速往返切换；
6. 姿态估计重置；
7. 目标位于限制范围外。

15. 验收标准

完成后的 TRACK 模式至少应满足：

1. 可通过遥控器模式开关进入和退出；
2. 目标位于图像中心时，飞行器不会持续产生姿态漂移；
3. 目标图像位置变化时，摄像头光轴向正确方向转动；
4. 油门杆只改变总推力，不改变目标指向逻辑；
5. Yaw 摇杆控制绕目标视线的 twist；
6. twist 操作时，摄像头光轴偏离目标的角度保持在允许范围内；
7. 模式切换时姿态和推力没有明显阶跃；
8. 目标丢失不会导致突然停转或推力归零；
9. 超时后能够可靠切回预设安全模式；
10. TRACK 下不存在两个模块竞争发布姿态设定的问题。

16. 物理约束说明

摄像头光轴与飞行器推力轴重合时：

摄像头指向目标
≈
推力方向指向目标

但飞行器实际加速度还受到重力影响，因此：

净加速度方向
≠
推力方向

除非目标恰好位于特定方向，飞行器不能同时严格满足：

- 摄像头始终精确指向目标；
- 净加速度始终精确指向目标；
- 高度保持不变。

第一版应优先实现：

视觉精确指向 + 手动总推力

允许实际轨迹受重力影响。

后续若获得目标距离、相对速度和飞行器位置，可增加重力补偿和轨迹控制，使视觉目标在图像中心附近小范围偏置，以换取更准确的接近轨迹。

17. 最终建议

推荐采用以下最终结构：

独立 NAVIGATION_STATE_TRACK
+
独立 track_control 模块
+
自定义 TrackTarget uORB 消息
+
输出 vehicle_attitude_setpoint
+
复用 PX4 原有姿态/角速度/控制分配

TRACK 不应建立在 Position 模式内部，也不应直接修改电机输出。

最关键的三项修改是：

1. 将 TRACK 正式接入 Commander 和遥控器模式槽；

2. 保证 TRACK 下只有 `track_control` 生成姿态设定；
3. 将 Yaw 摇杆定义为绕目标视线的 twist，而不是普通航向角。

该方案能够在尽量保留 PX4 原有稳定控制链的前提下，实现目标指向、手动推进和机身轴向旋转控制。